

- 1. Executive Summary
- 2. Repository Structure
- 3. System Architecture
- 4. Execution Flows
- 5. Personal Finance Audit
- 6. Finance Tutor Audit
- 7. Financial Tools Audit
- 8. LLM / AI Architecture
- 9. User State / Memory
- 10. Voice System
- 11. Talking Avatar
- 12. API Audit
- 13. Database / Storage
- 14. Dependency Audit
- 15. Code Quality
- 16. Security
- 17. Testing
- 18. Future: Finance News
- 19. Future: Equity Research
- 20. Feature Classification
- 21. Complexity Cut List
- 22. Recommended Architecture
- 23. Recommended Directory Layout
- 24. Migration Plan
- 25. SDE Value Assessment
- 26. AI/ML Value Assessment
- 27. Finance/Quant Value
- 28. Final Product Definition
- 29. Brutally Honest Verdict

ARIA: what it actually is, versus what it should stay

A full trace of the repository as it exists today — every claim below is backed by a file and, where useful, a line number. No refactors, no deletions, no new code. Just the truth of the system, and where the complexity earns its keep.

Repo ARIA_CLEAN **Branch** chore/production-readiness **Python files audited** ~65

Method full read of every module + 2 live runtime probes

01

Executive Summary

ARIA is further along than the brief for this audit assumes, and its worst problem isn't over-engineering — it's **disconnection**. The deterministic finance tools are genuinely excellent. Two of the six trained ML models are real, correctly wired, and still functionally frozen in the live product because a UI feature to feed them real interaction data was never built. A handful of files are dead weight left over from an earlier scaffold. And the avatar — the thing this audit was nominally worried about being over-built with Wav2Lip/SadTalker/LivePortrait stacks — is in fact the single most disciplined piece of the whole codebase.

The central finding, stated once here because it recurs through half of this document: **Module 1 (finance) and Module 2 (tutor) each contain a working, trained, correctly-orchestrated ML component that the live chat UI never actually feeds.** Module 1's LSTM spend-forecaster and Isolation-Forest anomaly detector need transaction history; the chat pipeline always calls them with an empty transaction list (`modules/finance/pipeline.py:232`), because there is no UI flow to enter transactions. Module 2's DKT mastery tracker and DQN teaching-policy need quiz-answer history; the chat pipeline generates a real quiz item via a real LLM call whenever the DQN chooses `ask_quiz`, and the Streamlit UI never renders it or wires an answer back to `modules/tutor/orchestrator.py:76` `record_answer()` — confirmed by grep, there is no such call anywhere in `interface/streamlit_app.py`. Both models are real thesis-grade work sitting one missing widget away from actually mattering.

Everything else is closer to what you'd hope: the XGBoost risk model, the LP budget optimizer, the tax engine, the SIP/EMI calculators, Module 3 (market) and Module 4 (equity research) all run on real data through real code paths every single time. The LLM is used correctly — for narration and light extraction, never for arithmetic. The test suite (50 tests) mocks the LLM properly and tests real deterministic behavior, not implementation details. And the avatar is a free, self-hosted, browser-native 3D lip-sync system with no GPU dependency and no API key, exactly matching your stated product vision — the Tavus CVI integration

(added and hardened this session) is a clearly-marked, opt-in, paid alternative sitting behind its own UI toggle, not a replacement.

The cut list in §21 is short and concrete: two genuinely broken files, two dead stubs, one dead general-purpose module, and one config block that documents a device this project never actually asks a GPU to touch. Nothing on it is a trained model, a routing rule, or a UI surface. The bigger lift this audit surfaces isn't cutting — it's closing the two feedback loops above, which is design and UI work, not new ML.

6 real trained ML models — 4 fully live, 2 wired but cold-start	2 broken / non-importable files found	4 fully dead files (zero importers)	50 passing pytest tests, LLM always mocked	0 LivePortrait / Wav2Lik / MuseTalk / SadTalker code present
---	---	---	--	--

02

Current Repository Structure

The tree is already close to the "clean" shape this audit's brief asks for — it just isn't labeled that way, and a few directories carry vestigial content from before the Stage 0–7 rewrite (per your own roadmap notes). Below, every top-level directory, its real responsibility, and whether that responsibility is the right one.

DIRECTORY	ACTUAL CONTENTS	VERDICT
<code>core/</code>	<code>router.py</code> (M3/M4 precise predicates + coarse keyword bucket), <code>orchestrator.py</code> (single <code>handle_query</code> dispatcher), <code>session.py</code> (in-memory chat history)	Correctly scoped — this is the layer the brief asks for, just no
<code>modules/finance tutor market equity_research/</code>	One package per BTP module, each with its own <code>pipeline.py</code> entry point	Right shape. Finance and tuto dead <code>utils.py</code> (§21).
<code>ai/llm/</code> , <code>ai/speech/</code>	Groq+Gemini client, prompt templates, Whisper STT, edge-tts/pyttsx3 TTS	Correctly the shared AI layer – import from here, nothing dup
<code>shared/</code>	SQLite user store, FAISS vector store, company resolver, intent classifier, NER, one dead file	Right idea, one file (<code>data_loa</code> scaffold debris.
<code>interface/</code>	Streamlit app (single file, ~500 lines) + <code>avatar/</code> subpackage (TalkingHead + Tavus CVI)	The presentation layer is corre business logic — <code>streamLit_</code> contains finance math or tuto <code>_handle_query</code> dispatch + r

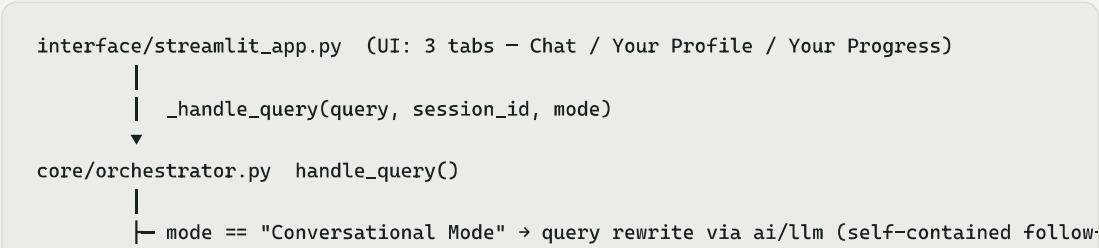
DIRECTORY	ACTUAL CONTENTS	VERDICT
<code>config/</code>	<code>paths.py</code> (filesystem), <code>settings.py</code> (runtime env), <code>model_config.py</code> (hyperparameters)	Textbook three-way split. One config block inside <code>model_co</code>
<code>models/</code>	<code>finance/</code> and <code>tutor/</code> hold real trained artifacts; <code>intent_classifier/</code> , <code>ner_extractor/</code> , <code>embeddings/</code> , <code>llm/</code> are empty directories	The four empty dirs are the pl an earlier, abandoned plan (a classifier + NER model) that w keyword heuristics without re scaffolding.
<code>tests/</code>	Mirrors <code>modules/</code> , plus <code>test_router.py</code> and <code>test_adversarial.py</code>	Good shape, good discipline (
<code>scripts/</code>	One-off data/index/model generators, invoked via <code>python -m</code>	Correctly kept out of the runti
<code>data/</code>	Synthetic training data, FAISS index, SQLite DB, a stray <code>data/bin/ffmpeg.exe</code> copy	Fine; gitignored appropriately
<code>reports/</code> , <code>ARIA Requirements.pdf</code>	Generated project report + the original BTP spec PDF	Documentation, not code — r opinion needed.
<code>main.py</code> (root)	A 20-line <code>argparse</code> stub that prints "not implemented"	Dead. The real entrypoint is <code>streamlit run interface/s</code> documented correctly in <code>REA</code> <code>main.py</code> is scaffold debris no See §21 .

No duplicate directories, no abandoned parallel-implementation folders (e.g. an old `api/` next to a new one). The debris that exists is all at the *file* level inside otherwise well-placed directories, not at the directory level — which is the easier problem to have.

03

Current System Architecture

There is no FastAPI/Flask backend behind the core product — Streamlit *is* the whole application layer, calling Python functions directly. This is a legitimate, deliberate choice for a single-process demo app, not a shortcut that needs justifying — see [§12](#). The one place a real HTTP server exists is the optional Tavus CVI bridge, which is intentionally separate.



```

|
├ core/router.py: is_equity_research_query() → modules/equity_research/pipeline.py
├ core/router.py: is_broad_market_query() → modules/market/pipeline.py
├ core/router.py: route_query() == "finance" → modules/finance/pipeline.py
├ core/router.py: route_query() == "tutor" → modules/tutor/pipeline.py
└ else → generic ai/llm/groq_client

```

Every module pipeline funnels through `ai/llm/groq_client.generate_response()`
 (Groq openai/gpt-oss-120b → Gemini gemini-3.6-flash fallback, both wrapped, never ...)

`core/session.py` - ephemeral, in-process chat turn history (lost on restart)
`shared/user_store.py` - persistent SQLite: FinancialProfile (M1), LearningState (M2)

The one deviation from a clean layered model: `core/router.py` 's precise Module 3/4 predicates (`is_equity_research_query` , `is_broad_market_query`) are checked *before* the coarse `route_query()` bucket, with a comment at `core/orchestrator.py:47-52` explaining exactly why (the coarse keyword classifier's `MARKET_KEYWORDS` list doesn't cover terms like "nifty"/"sensex"). This isn't architectural debt — it's a documented, deliberate ordering fix. Flagged here only because the brief asked to identify "incorrectly coupled" flows, and this is the one place routing logic isn't a single clean waterfall; it is, however, correctly explained in the code itself.

04

Request / Data / Execution Flows

Text request flow

`streamlit_app.py` text input → `_handle_query()` (reloads pipeline modules so local edits apply without restarting Streamlit, then calls) → `core/orchestrator.handle_query()` → router dispatch → module pipeline → `ai/llm/groq_client.generate_response()` for narration → `render_response()` parses the `Insight/Analysis/Recommendation/Risk` (or tutor's `Cognitive/Functional/Causal`) sections into styled cards. This is a real, traceable, single path — not the deep multi-hop flow the brief speculatively diagrammed.

Voice request flow

`st.audio_input` → `ai/speech/stt.transcribe()` (Whisper) → the transcript is written into the same text box (`st.session_state.query_text`) → from there it's identical to the text flow. Voice is a transcription front-end bolted onto the text pipeline, not a parallel system — which is the right design, not a shortcut.

Finance Tutor flow — as designed vs. as it runs

As designed (`modules/tutor/orchestrator.py:31-73`): query → SBERT+FAISS concept retrieval → DKT mastery lookup → DQN teaching-action choice → level-appropriate TAXAL explanation → optionally a quiz → user answers → `record_answer()` updates mastery → next turn reflects it.

As it actually runs in the live Streamlit app: query → retrieval → DKT (always reads empty history, so always returns all-zero mastery) → DQN (evaluates a state that never changes) → explanation at level 1 → quiz generated by a real LLM call →

quiz silently discarded (confirmed: no `"quiz"` key is ever read anywhere in `streamlit_app.py`) → the loop never closes. See [§6](#) for the full trace.

Personal Finance flow — as designed vs. as it runs

As designed (`modules/finance/orchestrator.py:32–118`): profile + transaction history → XGBoost risk → LSTM 3-month category forecast → Isolation Forest anomaly scan → LP budget optimization → SIP plan → tax → LLM narration grounded in all of the above.

As it actually runs: `modules/finance/pipeline.py:232` always constructs `UserFinancialInput( ... , transactions=[])` — there is no chat flow, form, or upload path anywhere in the UI that ever populates transaction history. XGBoost still gets a meaningful prediction (its profile-only features — income, age, dependents, debt-to-income, city tier — are always populated; only the transaction-derived features zero out, per the explicit comment at `modules/finance/features.py:52–58`). LSTM forecast and Isolation Forest both explicitly early-return degenerate output when there's no transaction data (`forecast_model.py:150–159` returns all-zero forecasts; `anomaly.py:23, 55–56`'s `MIN_TRANSACTIONS = 10` guard means `detect_for_user` always returns `[]`).

Avatar flow

Text response → `ai/llm/audio_script.generate_audio_script()` (a second, short LLM call that rewrites the answer into a 20–30s spoken script) → `ai/speech/tts.synthesize()` (edge-tts, pyttsx3 fallback) → `interface/avatar/render.build_avatar_html()` base64-embeds the audio into a self-contained HTML page → `st.components.v1.html()` → the browser's WebGL context runs TalkingHead's phoneme-driven lip-sync entirely client-side. No server-side video generation, no GPU, no per-frame rendering pipeline — see [§11](#).

Broad-market / equity data flow (already real, not hypothetical)

The brief frames financial news and equity research as future work with only an interface to design for. Both already exist as working pipelines: `modules/market/analyzer.py` pulls live Nifty/Sensex/sector data via yfinance plus market-wide headlines via Economic Times' public RSS feed (no API key — explicitly chosen over NewsAPI's free tier, which forbids this use per the comment at `analyzer.py:1–9`); `modules/equity_research/` pulls per-company fundamentals by scraping screener.in and live price/news via yfinance. See [§18](#) and [§19](#).

05

Personal Finance Assistant Audit

Financial understanding & personalization

Persisted per-user in `shared/user_store.py`'s `FinancialProfile` table: income, age, dependents, existing EMI, emergency-fund months, city tier, tax regime, plus the latest risk label/confidence. This is genuinely minimal and well-chosen — no

over-collection. Extraction from free text is a deliberately layered regex approach in `modules/finance/pipeline.py : _parse_indian_amount()` (lines 86–117) handles lakh/crore/k Indian numeric shorthand with word-boundary guards specifically added after two documented false-positive bugs (`"emi"` matching inside `"premium"` , `"earn"` matching inside `"learn"` — see the inline comment at line 92-94), and `_try_build_financial_profile()` (lines 200–232) merges new-turn values with the saved profile using explicit `is not None` checks (not truthy checks) so a legitimate "I earn 0 now" doesn't get silently discarded by a falsy-value bug — also explicitly commented (lines 208-209, 215-217). This is *the correct approach* for a system this size: pure regex, deliberately, with the reasoning left in the code. No LLM-based structured extraction exists or is needed here — the domain (Indian salary/EMI phrasing) is narrow enough that regex is both more reliable and drastically cheaper than a model call per turn.

Financial calculations — deterministic, correctly separated from the LLM

Every number a user sees is computed in plain Python before the LLM ever runs, and the LLM's system prompt for narration explicitly forbids inventing values (`M1_NARRATION` in `ai/llm/prompt_templates.py:43-82` : "Use ONLY the numbers provided below. Do NOT invent any values."). This is exactly the principle the brief asks to verify — it already holds:

- **Risk profile** — real trained XGBoost classifier, not a rule ([§7](#))
- **Budget** — real linear program via `scipy.optimize.linprog` , not the 50/30/20 rule as a hardcoded split (`modules/finance/budget_optimize.py`) — the 50/30/20 rule is explicitly the *starting point* the LP is built to override when infeasible, per the module docstring
- **Tax** — closed-form Indian slab computation, both regimes, with Section 87A rebate and marginal-relief smoothing (`modules/finance/tax.py:57-68`) — slabs sourced and dated live against four tax sites (comment at lines 4-9)
- **SIP/EMI** — standard closed-form annuity formulas (`modules/finance/sip_emi_calc.py`)

None of this is embedded in prompts. The LLM's only job on the numeric side is turning a filled-in template into readable Indian-English prose.

Risk profiling, specifically

Real trained XGBoost multiclass classifier (`modules/finance/risk_model.py`), not a hardcoded formula — five profile features plus (when available) sixteen transaction-derived features (`features.py:15-32`), SHAP-explained top contributing factors per prediction (with a documented never-raise fallback if SHAP itself fails, `risk_model.py:49-65`). Per your project roadmap notes, this scored 0.99 macro-F1 in 5-fold CV on synthetic data. It is deterministic given its inputs, personalized to whatever profile fields exist, and — critically — is the *one* of the three finance ML models that stays meaningful even with zero transaction history, because its most important features (income, age, EMI-to-income) are *always* populated. This is the finance module's best-designed piece.

CENTRAL FINDING – RESTATED WITH FULL EVIDENCE

The LSTM spend forecaster and Isolation Forest anomaly detector are **real, correctly trained, correctly wired** — and functionally inert in the live product,

because nothing in the Streamlit UI ever collects a user's transaction history.
`modules/finance/pipeline.py:232` hardcodes `transactions=[]` on every call.
`forecast_model.py:150` 's own docstring says it plainly: "Falls back to an all-zero forecast (no ML) if there's no transaction history at all." `anomaly.py:23` 's
`MIN_TRANSACTIONS = 10` guard means `detect_for_user` returns `[]`
unconditionally in the live app. Both models train and evaluate correctly offline (per your roadmap: LSTM MAE 2.3% of income, Isolation Forest caught 15/15 injected anomalies on synthetic data) — the gap is entirely in the interface layer, not the models. See [§21](#) for the fix, which is a UI feature, not a model change.

06

Finance Tutor Audit

One correction to the brief's own framing before the audit: what's implemented is **Deep Knowledge Tracing** (Piech et al. 2015 — an LSTM over one-hot concept/correctness sequences, `modules/tutor/knowledge.py:20–44`), not classical **Bayesian** Knowledge Tracing (which would be a per-concept 4-parameter HMM). They solve the same problem — mastery estimation from an interaction sequence — but DKT is the more modern, higher-capacity approach and is what the code, the class name, and the training script (`modules/tutor/train/train_lstm_kt.py`) actually implement. Worth knowing precisely, since "BKT" and "DKT" get used interchangeably in casual conversation about this kind of system but are different models with different guarantees.

Retrieval

Real Sentence-BERT (`all-MiniLM-L6-v2`) + FAISS cosine search over a 50-concept knowledge base (`modules/tutor/retriever.py`), gated by an empirically-tuned similarity threshold (`config/settings.py:79–82`: "genuinely off-topic queries score ≤ 0.23 , misspelled-but-real finance queries score ≥ 0.41 " — a threshold chosen from data, not guessed). Below threshold, the pipeline correctly falls through to a generic tutor prompt instead of forcing a bad concept match. This is real, live, and load-bearing on every tutor query — no gap here.

DKT (mastery)

Real trained LSTM, correctly called from `modules/tutor/orchestrator.py:40` on every tutor turn. **But** its input is the user's persisted interaction history (`shared/user_store.get_learning_state`), and that history can only grow via `record_answer()` — which nothing in `interface/streamlit_app.py` ever calls (confirmed by grep across the whole file: zero matches for `record_answer`). Every real user's mastery vector is permanently the cold-start default (`knowledge.py:101–104`: all-zero when `history` is empty) in the shipped product today.

DQN (teaching-action selection) — real RL, not decorative, but starved of state

This is genuinely a trained `stable_baselines3.DQN` policy (`modules/tutor/teaching.py`), trained via self-play against a simulated tutoring environment (no external dataset —

`modules/tutor/train/train_dqn_teacher.py`), evaluated to a mean reward of 3.75 against a >0 target per your roadmap notes. State: `[mastery_target, avg_mastery_prereqs, last_5_accuracy, session_length, concept_difficulty]`; six actions (simplify / example / quiz / deepen / analogy / prerequisite). This is a real, non-trivial application of RL to an actual decision (not an RL system bolted on for demo value with no real choice attached to it) — the honest caveat is that because `mastery_target` and `last_5_accuracy` never move off their cold-start defaults in the live app (see above), the policy is currently always evaluating close to the same state vector, so its *trained variety* of behavior is not visible to a real user, even though the policy itself is real and was genuinely trained to handle a range of states.

REBUILD, NOT REMOVE

The fix here is small relative to the models it unlocks: a quiz-answer widget in the Chat tab (radio buttons over `QuizItem.correct_answer` + `wrong_answers`, already fully generated and already flowing through the response dict at `modules/tutor/pipeline.py:38`) that calls `modules.tutor.orchestrator.record_answer(user_id, concept_id, is_correct)` on submit. That one widget makes DKT and DQN — both already real, already trained, already wired everywhere else — actually observable and actually adaptive for a real user. This is the single highest-leverage change surfaced by this entire audit.

TAXAL explanation & quiz generation

Both are thin, well-scoped LLM calls with strict parsing (`modules/tutor/explainer.py`, `modules/tutor/quiz.py`) — level-appropriate (1–8) via `TAXAL_LEVEL_HINTS` / `TAXAL_LENGTH` dictionaries in `ai/llm/prompt_templates.py:17–37`, further styled by the DQN's chosen action via `TAXAL_ACTION_STYLE_HINTS` (lines 110–117). `quiz.py`'s parser returns `None` on malformed LLM output rather than crashing or fabricating a quiz — good defensive design.

Financial Tools Audit

TOOL	FILE	DETERMINISTIC?	TESTED?
Risk profile	<code>modules/finance/risk_model.py</code>	Given inputs, yes (trained XGBoost, no randomness at inference)	Yes — <code>tests/finance/test_risk_model.</code>
Budget (LP)	<code>modules/finance/budget_optimize.py</code>	Yes — <code>scipy.linprog</code>	Yes — <code>tests/finance/test_budget_opti</code>

TOOL	FILE	DETERMINISTIC?	TESTED?
Tax (old/new regime)	<code>modules/finance/tax.py</code>	Yes — closed- form slabs	Yes — <code>tests/finance/test_tax.p</code>
SIP future value / required	<code>modules/finance/sip_emi_calc.py</code>	Yes — closed- form annuity math	Yes — <code>tests/finance/test_sip_emi_cal</code>
EMI + total interest	<code>modules/finance/sip_emi_calc.py</code>	Yes — reducing- balance formula	Yes (same file)
Spend forecast	<code>modules/finance/forecast_model.py</code>	Yes, given a sequence — but LSTM never sees real sequences live	Yes — <code>tests/finance/test_forecast_mo</code>
Anomaly detection	<code>modules/finance/anomaly.py</code>	Yes, per-user fit — but never fit live (needs 10+ txns)	Yes — <code>tests/finance/test_anoma</code>

Every tool in this table is unit-tested against real numeric expectations, not mocked math — this is the correct testing posture for financial calculations specifically (see §17). No calculation anywhere in the finance module is delegated to the LLM.

08

LLM / AI Architecture Audit

One LLM client, one call surface, used everywhere:

`ai/llm/groq_client.generate_response(prompt, system_prompt)`. Every one of the four modules imports this single function — there is no per-module LLM wrapper duplication, no scattered client instantiation. This is the "LLM abstraction → provider → model" shape the brief asks for, already built, just without formal provider classes: a plain function with an internal two-tuple `_BACKENDS` list (`groq_client.py:129`) that tries Groq then Gemini in order.

Provider & fallback design

Two providers, not more (no Ollama, no local FinGPT, no third cloud provider — the original PDF plan's local-LLM idea was explicitly and correctly dropped per your roadmap notes, since it needed ~14GB VRAM this project doesn't have and can't assume for public deployment). The fallback chain is exactly two deep, wrapped so a caller can never see an unhandled exception — `generate_response` always returns a string, prefixed `"Error: ... "` on total failure, which `interface/streamlit_app.py:116–125`'s `render_response` specifically detects and turns into a clear user-facing banner instead of leaking a raw error string into the chat. This is not excessive fallback logic — it's the minimum needed for "a free-

tier API might rate-limit" to not mean "the demo breaks," and it's exactly two providers, not a chain of five.

One real, current bug found and fixed live during this session (not part of this audit's read-only scope, but material to note): both `DEFAULT_MODEL_NAME` (Groq) and `DEFAULT_GEMINI_MODEL` had drifted to retired model IDs as of today. Both providers' hosted catalogs move fast enough that hardcoded model-ID defaults are a genuine, recurring maintenance surface — worth a periodic live check, not a one-time fix.

Is this actually agentic?

Partially, and it says so honestly rather than dressing up prompt engineering as agency. What's genuinely present: real tool use (deterministic finance calculators, real yfinance/screener.in data fetches, real vector retrieval), real state (SQLite profile + mastery, session chat history), and one real closed-loop decision system (the tutor's DQN choosing an action from live-ish state). What's *not* present: the LLM itself never chooses which tool to call — that decision is made entirely by `core/router.py`'s deterministic keyword/regex predicates before the LLM is ever invoked. There's no LLM-driven planning step, no ReAct-style loop, no LLM function-calling. This is the right architecture for this product (a routed, tool-augmented assistant), and it's honest to describe it as **router + tools + state**, not **agent** in the tool-calling-loop sense — using the more inflated label wouldn't survive a technical interview follow-up question, and the simpler true description is more defensible, not less impressive.

09

User State / Memory Audit

Two genuinely different, correctly separated memory systems exist — worth naming explicitly since nothing in the code currently calls out the split as a design decision:

PERSISTENT — SHARED/USER_STORE.PY

SQLite (`FinancialProfile` , `LearningState`), keyed by session/user id. Survives restarts. Holds exactly the minimal fields the brief asks for: income, age, dependents, EMI, emergency-fund months, city tier, tax regime, risk label (M1); per-concept mastery + interaction history (M2). No overcollection.

EPHEMERAL — CORE/SESSION.PY

A single in-process `dict[str, dict]` (`_SESSIONS`). Chat turn history only. Lost on every restart, and — because it's keyed purely by whatever `session_id` a caller passes — shared across any two callers that happen to pass the same id. In the main Streamlit UI this is safe: each browser tab gets a fresh `uuid4()` (`streamlit_app.py:480–481` , with a comment confirming this is deliberate for concurrent visitors). It was *not* safe in the Tavus bridge before this session's fix, which hardcoded a shared `session_id="tavus"` for every conversation — now changed to read history from the request's own message list instead of touching this store at all.

Minimal state is already the reality here, not a target to migrate toward. The one thing worth documenting (not fixing) is that ephemeral chat history and persistent structured profile were never explicitly designed as a two-tier memory system — they just ended up that way, correctly, through two different modules solving two different problems.

10

Voice System Audit

LAYER	PRIMARY	FALLBACK	NOTES
STT	OpenAI Whisper (<code>base.en</code> , local) — <code>ai/speech/stt.py</code>	None	Single engine, not faster-whisper. CUDA auto-detected, safe CPU fallback (lines 111-124). Runs synchronously in the Streamlit request thread — acceptable at demo scale, would need a job queue at real scale.
TTS	edge-tts (neural, <code>en-IN-NeerjaNeural</code>) — <code>ai/speech/tts.py</code>	pyttsx3 (fully offline)	Clean two-tier design: async edge-tts call wrapped in a thread with a hard timeout (<code>_run_edge_tts</code> , lines 57-88), falling back to pyttsx3 only on failure. No Coqui TTS — correctly not needed given edge-tts already covers "high quality, free, no API key."

CODE QUALITY – PORTABILITY

`ai/speech/stt.py:82-86` hardcodes a Windows path under a different developer's user account into the ffmpeg-discovery candidate list:
`C:\Users\jtmah\AppData\Local\ ... \ffmpeg-8.1-full_build\bin` . Harmless (it's just one more candidate that will never exist on your machine and gets skipped), but it's a leftover from a copied snippet and won't work on any machine but that one — worth removing on the next pass through this file.

Both engines are lazy-loaded, thread-safe (STT uses a module-level lock, line 51), and have real `unload()` /health-check functions. Overall this is the simplest robust design available at this project's scale — no unnecessary redundancy, no missing fallback.

11

Realistic Talking Avatar Audit

This is the part of the brief most worth correcting against the actual repository: there is no LivePortrait, MuseTalk, Wav2Lip, or SadTalker anywhere in this codebase, and there never has been per the git history. The avatar is **TalkingHead** — a browser-native, WebGL-rendered, VRM-rigged 3D model with phoneme-driven lip-sync, loaded from jsdelivr's CDN at runtime (MIT licensed, no API key — `interface/avatar/NOTICE.md`). This is a fundamentally different, and for this project's constraints, correct category of technology: not video generation, not a diffusion or GAN-based talking-head model, but a real-time 3D rig animated by audio — closer to VTuber tooling than to deepfake-style avatar synthesis.

TECHNOLOGY	PRESENT IN REPO?	WORKING?	LATENCY	GPU?	VERDICT
TalkingHead + HeadAudio (3D VRM, browser)	Yes — <code>interface/avatar/</code>	Yes — verified live this session	Near-instant; audio embedded as base64, rendered client-side	No — runs on the viewer's browser GPU/WebGL, not the server	KEEP
Tavus CVI (WebRTC video avatar, paid API)	Yes, opt-in — <code>interface/avatar/tavus_cvi.py</code>	Yes — provisioning, callback endpoint, and conversation teardown all verified live this session	~10s provisioning, then real-time video call	No local GPU — runs on Tavus's cloud	OPTIC
LivePortrait / MuseTalk / Wav2Lip / SadTalker	No	n/a	n/a	n/a	NOT F

Real-time classification

TalkingHead is genuine real-time streaming, not pre-generated or generate-on-demand — the browser animates the 3D rig live against the audio waveform the moment it's embedded; there is no server-side video render step at all, which is precisely why it needs no GPU and imposes no per-response latency cost beyond LLM narration + TTS synthesis. This is the most practical possible answer to "realistic, convincing, demonstrable, not maximum model complexity" — the brief's own stated goal — and it already exists exactly as specced.

Modularity

`interface/avatar/render.py` 's `build_avatar_html()` takes only a response's audio path as input and returns self-contained HTML; it has zero imports from `modules/`, `core/`, or `shared/`. Swapping the avatar out would touch only `interface/streamlit_app.py` 's rendering branch (currently gated on `turn.get("mode") == "Conversational Mode"`, `streamlit_app.py:400`) — finance logic, tutor logic, and the LLM layer are provably untouched by an avatar swap, which is exactly the modularity principle the brief asks to verify. The Tavus integration proves this in practice: it was added as a third, fully independent mode (`"Tavus CVI Mode (WebRTC)"`) without modifying a single line of `core/` or `modules/`.

API Audit

There is no REST API surface for the core product — Streamlit calls Python functions in-process. This is not a gap to fill; a separate API layer would add a network hop and a serialization boundary for zero benefit to a single-process demo app, and nothing about the brief's stated goals (personal use, thesis defense, eventual Streamlit Cloud deploy) needs one. The only real HTTP server in the

repository is the Tavus custom-LLM bridge, which exists specifically because Tavus's cloud needs a public URL to call into — a genuine, external constraint, not an architectural choice made for its own sake.

ENDPOINT	FILE	PURPOSE	CALLER	VI
POST /v1/chat/completions	interface/avatar/tavus_llm_server.py	OpenAI-compatible bridge — Tavus's cloud calls this to get ARIA's answer for a spoken turn	Tavus's servers, via the ngrok tunnel	ve th (2 co T, fo re
GET /healthz	same file	Liveness probe, added this session	Not yet called by anything — added defensively	

No duplicated endpoints, no dead routes, no versioning inconsistency — because there are only two routes total, both single-purpose, both scoped to the one feature that genuinely needs a network-reachable server.

Database / Storage Audit

STORE	TECHNOLOGY	CONTENTS	NOTES
User profile + learning state	SQLite, SQLAlchemy ORM (<code>shared/user_store.py</code>)	<code>financial_profiles</code> , <code>learning_state</code> tables	Two tables, both minimal, both correctly normalized to their module. No ORM session leaks to callers — every function is dict-in/dict-out (module docstring, lines 10-13).
Concept vectors	FAISS flat index + JSON metadata (<code>shared/vector_store.py</code>)	<code>data/vector_store/concepts/</code>	Namespaced by design for future reuse by other modules — not over-engineered, just deliberately not hardcoded to one caller.
Chat history	In-process dict (<code>core/session.py</code>)	Turn-by-turn Q&A, ephemeral	Correctly not persisted — this is working memory, not a record that needs to survive a restart.
Generated audio	OS temp files (<code>tempfile.NamedTemporaryFile</code>)	TTS <code>.mp3</code> / <code>.wav</code> per response	Correctly not persisted — never written into the repo or a permanent data dir.

STORE	TECHNOLOGY	CONTENTS	NOTES
Tavus runtime files	Plain text files (<code>ngrok_url.txt</code> , <code>.tavus_persona_cache.json</code>)	Tunnel URL + cached persona id	Added/gitignored this session — regenerated per run, not committed.

Nothing here is over-persisted, and nothing that should persist is missing. The one gap is upstream of storage, not in it: there's no transaction-history table for Module 1 to write into even if a UI existed to collect it (see [§21](#)'s recommended fix, which would need one).

Dependency Audit

DEPENDENCY	USED FOR	JUSTIFIED?
<code>xgboost</code> , <code>shap</code>	Module 1 risk model + explanations	Yes — real trained model, real per-prediction explanation
<code>torch</code>	LSTM spend forecast, DKT, Whisper	Yes, though it's the single heaviest dependency in the stack and is pulled in by three unrelated features — worth knowing it's not optional even if you cut the two ML models it backs, since Whisper needs it too
<code>scipy</code>	LP budget optimizer	Yes — one function, <code>linprog</code> , genuinely load-bearing
<code>faiss-cpu</code> , <code>sentence-transformers</code>	Concept retrieval	Yes — real semantic search, real threshold-gated confidence
<code>gymnasium</code> , <code>stable-baselines3</code>	DQN teaching policy training + inference	Yes, real RL — see caveat in §6 about the policy's state currently being starved, not about the dependency being unjustified
<code>groq</code> , <code>google-genai</code>	LLM narration, two providers	Yes — both actively used, no redundant third provider
<code>openai-whisper</code> , <code>imageio-ffmpeg</code>	STT	Yes
<code>edge-tts</code> , <code>pyttsx3</code>	TTS primary + fallback	Yes, both actually reachable (§10)
<code>yfinance</code> , <code>beautifulsoup4</code> , <code>requests</code>	Modules 3/4 live data	Yes
<code>sqlalchemy</code>	User store ORM	Yes, appropriately light (two tables)
<code>streamlit</code>	The entire UI/app framework	Yes
<code>fastapi</code> , <code>pyngrok</code> , <code>uvicorn</code>	Tavus CVI bridge only	Justified <i>only if you keep Tavus mode</i> — these three exist for one optional, paid feature and nothing else in the app touches them. If Tavus mode is ever dropped, these three drop with it cleanly (no other module imports them).

No duplicate libraries solving the same problem (e.g. two HTTP clients, two ORMs, two vector-search libraries). `pyproject.toml` currently only holds pytest config — there's no `[project]` dependency declaration, so `pip install -e .` isn't

supported; `requirements.txt` is the sole source of truth for install. Not a defect at this scale, just worth knowing if packaging ever becomes a goal.

15

Code Quality Audit

FILE : LINE	ISSUE
<code>modules/finance/utils.py:6</code>	Imports <code>RiskPrediction</code> from <code>modules.finance.risk_model</code> — that name does not exist in that module (verified: <code>python -c "import modules.finance.utils"</code> raises <code>ImportError</code> immediately). The whole file cannot be imported.
<code>modules/finance/utils.py</code> (whole file)	Duplicates <code>BudgetAllocation</code> and <code>SipPlan</code> with different field shapes than the real pydantic schemas in <code>modules/finance/schemas.py</code> , and duplicates budget/SIP logic with simpler, non-LP, hardcoded-ratio math than the real <code>budget_optimize.py</code> / <code>orchestrator.py</code> . Zero importers anywhere in the codebase (grep-confirmed).
<code>modules/tutor/utils.py</code>	<code>load_kt_sequences()</code> always returns <code>[]</code> . Zero importers.
<code>shared/data_loader.py</code>	Placeholder <code>FinancialProfile</code> dataclass (name-collides with the real SQLAlchemy <code>FinancialProfile</code> in <code>user_store.py</code>) + stub <code>load_mock_data</code> / <code>load_csv</code> . Zero importers anywhere in source (only appears in the generated PDF/MD report).
<code>interface/components.py</code>	One dead <code>header()</code> helper. Zero importers — the real rendering (<code>render_response</code> , <code>cards</code> , etc.) all lives directly in <code>streamlit_app.py</code> .
<code>main.py</code> (root)	Argparse scaffold that prints "not implemented." Not the real entrypoint (that's <code>streamlit run interface/streamlit_app.py</code> , correctly documented in <code>README.md</code>).
<code>config/model_config.py:18–35</code>	<code>XGB_PARAMS</code> hardcodes <code>"device": "cuda"</code> . Never used directly — only <code>XGB_CPU_PARAMS</code> (a copy with <code>device</code> overridden) is imported by the training script (confirmed: <code>train_xgb_risk.py:17,60</code>). Behavior is correct; the dead block just contradicts the README's explicit "no GPU needed anywhere in this project" claim on a read-through.
<code>modules/equity_research/screener_adapter.py:63,68</code>	<code>requests.get(url, headers=...)</code> with no <code>timeout</code> — a slow/hung screener.in response blocks the single Streamlit request thread indefinitely. (<code>modules/market/analyzer.py:107</code> 's equivalent call correctly sets <code>timeout=8</code> — inconsistent between two very similar modules.)
<code>ai/speech/stt.py:82–86</code>	Hardcoded ffmpeg path under a different machine's username (<code>C:\Users\jtmah\...</code>) in the fallback candidate list.
<code>models/intent_classifier/</code> , <code>models/ner_extractor/</code> , <code>models/embeddings/</code> , <code>models/llm/</code>	Four empty model directories with no <code>.gitkeep</code> even, unlike <code>modules/finance/</code> and <code>modules/tutor/</code> which hold real artifacts.

Physical trace of an abandoned "train a real intent classifier + NER model" plan, superseded by the keyword/regex versions in `shared/intent_classifier.py` and `shared/ner.py` that are what's actually live.

Outside this list, the code quality bar is genuinely high: functions are short and single-purpose, docstrings explain *why* not just what (the false-positive-regex comments in `finance/pipeline.py` and the M1/M2 routing-order comment in `core/orchestrator.py` are good examples of exactly the kind of institutional-memory comment that prevents a bug from being silently reintroduced), and every ML component has an explicit `is_trained()` guard with a clear "train it first: `python -m ...`" error rather than crashing on a missing file.

16

Security Audit

- **Secrets handling** — real API keys live only in a gitignored `.env`; `.env.example` documents every key with no real values. No key is hardcoded in source. Good.
- **No timeout on outbound scraping calls** — `screener_adapter.py:63,68` (see §15). Not exploitable by a third party, but a legitimate availability risk (one slow upstream page hangs a user's session).
- **The Tavus FastAPI bridge has no authentication** (`tavus_llm_server.py`) — it binds `0.0.0.0:8000` and is tunneled publicly via ngrok for the duration of a session. Anyone who obtains the ngrok URL while it's live could POST to `/v1/chat/completions` and get free, unauthenticated completions routed through your Groq/Gemini keys. Low real-world risk given ngrok URLs are random and short-lived per session, but worth knowing this is an open relay while it's running, not a private one.
- **Web scraping fragility, not a security issue but adjacent** — `screener_adapter.py` parses screener.in's HTML structure directly (`section_id="profit-loss"` etc.); any layout change on their end silently degrades to empty data (handled gracefully — `get_screener_data` catches and returns `{"error": ...}`) — but worth knowing this is inherently a brittle integration, structurally, not a bug).
- **No PII logging found** — `loguru` calls throughout log query text and computed numbers, not names/emails/financial account identifiers, since none of those are ever collected in the first place.
- **CORS / auth** — not applicable to the Streamlit app itself (no separate frontend origin); not configured on the Tavus bridge, consistent with the "no auth" point above.

17

Testing Audit

50passing tests, verified
live this session**10**finance + tutor unit test
files**1**router logic test file
(pure predicates, no
ML/LLM)**1**adversarial corpus
(crash-resistance, not
correctness)

The mocking strategy (`tests/conftest.py`) is worth calling out specifically as well-engineered, not just present: it patches `generate_response` in *every module that imported it by value* (nine call sites, explicitly enumerated with a comment explaining *why* a single source-module patch isn't sufficient — Python binds a local reference on `from x import y`). That's the kind of test-infrastructure detail that's easy to get subtly wrong (patch the source, tests silently still hit the real API) and here it's correct and documented.

Financial calculations are tested against real expected numbers (deterministic assertions), not snapshot/implementation-detail tests — exactly the bar the brief asks for. The adversarial suite (`test_adversarial.py`) tests a different, equally real property: that empty input, 2000-character input, off-topic input, and zero-income input never crash the pipeline, only that a well-formed response dict comes back — the right test for that concern, since narration *quality* under adversarial input isn't a property a unit test can meaningfully assert with a mocked LLM anyway.

Coverage gaps, honestly: no automated test touches `modules/market/` or `modules/equity_research/` directly (both are only exercised transitively through the adversarial suite's multi-intent test); no test covers STT, TTS, or the avatar render path (reasonable — these need real audio/browser fixtures, not unit tests); and the Tavus integration added this session has zero automated coverage — it was verified live/manually only, which was the right call for external-API-dependent code but is worth formalizing if the feature stays.

18

Future Finance News Architecture

This is not a green-field design problem — `modules/market/analyzer.py` already answers a version of "what happened in the market today":

`get_index_snapshot()` (Nifty/Sensex level + 5-day trend via yfinance),

`get_sector_snapshot()` (equal-weight sector basket trend), and

`get_market_news()` (live headlines via Economic Times' public RSS, explicitly chosen over NewsAPI because NewsAPI's free tier forbids this exact use — a real, considered decision, not an oversight). "What happened in finance today" and "how did the Indian stock market perform today" are already answerable today by Module 3, routed automatically via `core/router.is_broad_market_query()` .

What's genuinely future work: a daily-digest framing ("summarize today" rather than "answer this specific question"), and possibly a scheduled/cached daily snapshot instead of computing on every request (currently `lru_cache` d per-process, which resets on every Streamlit restart — fine for a demo, not for a always-on daily-briefing feature). Neither needs a new `FinanceNewsService` abstraction; `MarketAnalyzer` already is that service, under Module 3's name.

Future Equity Research Architecture

Same correction applies more strongly here: this isn't future work, it's a complete, working Module 4 today. "Analyze TCS" and "Compare HDFC Bank and ICICI Bank" both already route correctly — `shared/company_resolver.py` maps 25+ common Indian company names/aliases to screener.in tickers, `modules/equity_research/financial_pipeline.py` pulls real revenue/profit/debt/equity history from screener.in and grounds the LLM response strictly in those scraped numbers ("Use ONLY the numbers provided below," `financial_pipeline.py:134`), and `modules/equity_research/investment.py` separately handles live price/PE/market-cap/news via yfinance, with an LLM-assisted company-name-resolution fallback for names Yahoo's own search can't match directly (`investment.py:41-64`).

Genuine future extension, not currently present: side-by-side company *comparison* ("Compare HDFC Bank and ICICI Bank" today resolves to whichever single company the resolver matches first, not a two-company diff) — this would be a real, scoped addition: run `process_financial_query` twice and add a comparison-specific prompt, reusing every existing piece. No architectural change needed, just a new branch in `modules/equity_research/pipeline.py`.

Feature Classification

FEATURE	IMPLEMENTATION	LI
XGBoost risk profile	<code>modules/finance/risk_model.py</code>	Ye
LP budget optimizer	<code>modules/finance/budget_optimize.py</code>	Ye
Tax engine	<code>modules/finance/tax.py</code>	Ye
SIP/EMI calculators	<code>modules/finance/sip_emi_calc.py</code>	Ye
LSTM spend forecast	<code>modules/finance/forecast_model.py</code>	No tra
Isolation Forest anomalies	<code>modules/finance/anomaly.py</code>	No
SBERT+FAISS concept retrieval	<code>modules/tutor/retriever.py</code>	Ye
DKT mastery tracking	<code>modules/tutor/knowledge.py</code>	No an
DQN teaching policy	<code>modules/tutor/teaching.py</code>	Re va
TAXAL explanation + quiz gen	<code>modules/tutor/explainer.py</code> , <code>quiz.py</code>	Ye
Module 3 market data	<code>modules/market/analyzer.py</code>	Ye

FEATURE	IMPLEMENTATION	LI
Module 4 equity research	<code>modules/equity_research/</code>	Ye
Groq→Gemini LLM fallback	<code>ai/llm/groq_client.py</code>	Ye
Whisper STT + edge-tts/pyttsx3 TTS	<code>ai/speech/</code>	Ye
TalkingHead 3D avatar	<code>interface/avatar/</code>	Ye
Tavus CVI (paid)	<code>interface/avatar/tavus_cvi.py</code>	Ye
Keyword intent classifier	<code>shared/intent_classifier.py</code>	Ye
Regex NER	<code>shared/ner.py</code>	Ye
<code>modules/finance/utils.py</code>	—	No im
<code>modules/tutor/utils.py</code>	—	No
<code>shared/data_loader.py</code>	—	No
<code>interface/components.py</code>	—	No
<code>main.py</code> (root)	—	No en
Empty <code>models/intent_classifier ner_extractor embeddings llm/</code>	—	No

21

Complexity Cut List

Six items. Every one is either genuinely broken, genuinely dead, or genuinely misleading — nothing here is being cut for being "too advanced." The real lift this audit found isn't cutting; it's the two feedback-loop rebuilds in \$5/\$6, which *add* a small amount of UI code to make existing, already-trained models matter.

REMOVE `modules/finance/utils.py` saves: one file, one import-time crash risk

Imports a class that doesn't exist (`RiskPrediction` from `risk_model.py`) — cannot be imported at all today. Duplicates `BudgetAllocation` / `SipPlan` with different, simpler, non-LP math than the real orchestrator uses. Nothing imports it. Nothing replaces it because nothing needs to — `budget_optimize.py` and `orchestrator.py` already do its job correctly.

REMOVE `modules/tutor/utils.py` saves: one file

`load_kt_sequences()` always returns `[]`. Zero importers. Nothing replaces it — training data loading for the real DKT model lives correctly in `scripts/generate_kt_sequences.py` and `modules/tutor/train/`.

REMOVE `shared/data_loader.py` saves: one file, one confusing name collision

Its `FinancialProfile` dataclass shares a name with the real SQLAlchemy model in `user_store.py` — a genuine trap for a future reader who greps for "FinancialProfile" and finds two unrelated definitions. Zero importers in source.

REMOVE `interface/components.py` saves: one file

One unused `header()` helper. All real UI rendering already lives in `streamlit_app.py`.

REMOVE OR REPOINT `root` `main.py` saves: confusion for a first-time reader

Prints "not implemented" and isn't the real entrypoint. Either delete it, or reduce it to the one line that matters: instructing the reader to run `streamlit run interface/streamlit_app.py`, which is already what `README.md` correctly says.

SIMPLIFY **the dead CUDA block in** `config/model_config.py:18–34`

saves: one contradiction with the README's "no GPU" claim

`XGB_PARAMS` (with `"device": "cuda"`) is never imported anywhere; only the CPU-forced copy is. Either delete `XGB_PARAMS` and keep only the CPU version under its own name, or add a one-line comment explaining it's kept only as the base dict `XGB_CPU_PARAMS` is derived from.

EXPLICITLY NOT ON THIS LIST

The DQN teaching policy, the LSTM spend forecaster, and the Isolation Forest anomaly detector. All three are real, correctly trained, correctly wired into their orchestrators, and unit-tested. Their problem is a missing UI feed, covered in §5/§6/§20 as **REBUILD** — not a reason to remove genuine thesis-grade ML work that a small UI addition would make fully live.

Recommended Final Architecture

The honest recommendation is: keep the layering exactly as it is, because it already matches the target shape. What follows names the layers that already exist, so the shape is legible rather than implicit.

Interface Layer	<code>interface/streamlit_app.py</code> (text + voice input, avatar render)
Agent / Application Layer	<code>core/router.py</code> + <code>core/orchestrator.py</code> (intent → module dispatch)
Personal Finance Layer	<code>modules/finance/</code> (profile, XGBoost risk, budget, tax, SIP)
Education Layer	<code>modules/tutor/</code> (retrieval, DKT, DQN, TAXAL, quiz)
Market / Equity Layer	<code>modules/market/</code> , <code>modules/equity_research/</code> (already-live "future")
AI Layer	<code>ai/llm/</code> (Groq→Gemini, prompt templates)
Voice Layer	<code>ai/speech/</code> (Whisper, edge-tts/pyttsx3)
Avatar Layer	<code>interface/avatar/</code> (TalkingHead default, Tavus CVI opt-in)
Data Layer	<code>shared/user_store.py</code> (SQLite), <code>shared/vector_store.py</code> (FAISS)
Config Layer	<code>config/</code> (paths, settings, hyperparameters)

The one real architectural change worth making is not structural — it's closing the two loops from §5/§6 (transaction entry → forecast/anomaly; quiz answer →

Recommended Directory Structure

No renames or moves are recommended — the existing layout already matches the responsibilities described in §22. The table below documents what belongs in each directory going forward, as a reference for future additions, not a migration target.

DIRECTORY	BELONGS HERE	DOES NOT BELONG HERE
<code>core/</code>	Routing predicates, the single orchestrator, session state	Any module-specific business logic or calculation
<code>modules/<name>/</code>	That module's pipeline, orchestrator, schemas, calculators, trained-model wrappers	Cross-module utilities (those go in <code>shared/</code>) or LLM client code (that goes in <code>ai/llm/</code>)
<code>shared/</code>	Genuinely cross-module helpers actually imported by 2+ modules	Speculative "might be useful later" stubs — this is exactly how <code>data_loader.py</code> happened
<code>ai/</code>	Provider-agnostic LLM/speech clients, prompt templates	Any finance/tutor-specific business rule
<code>interface/</code>	Streamlit rendering, avatar embedding — presentation only	Calculation, routing, or persistence logic (verified clean today)
<code>config/</code>	Paths, runtime env, hyperparameters — three files, three concerns, don't merge them	Any dead/unused constant block (§21)

Migration Plan

No rewrite. Six small, independent, low-risk phases — most are single-file deletions with no dependents to break, verified by grep in this audit.

PHASE	ACTION	FILES TOUCHED	RISK
1	Freeze & document (this audit)	None	None
2	Delete the five dead/broken files from §21	<code>modules/finance/utils.py</code> , <code>modules/tutor/utils.py</code> , <code>shared/data_loader.py</code> , <code>interface/components.py</code> , root <code>main.py</code>	None — zero importers confirmed for each, verified live for the two broken ones
3	Remove the dead CUDA config block + empty <code>models/*</code> dirs	<code>config/model_config.py</code>	None

PHASE	ACTION	FILES TOUCHED	RISK
4	Add transaction-entry UI (form or CSV upload) in the Profile tab, writing to a new small SQLite table	<code>interface/streamlit_app.py</code> , <code>shared/user_store.py</code>	Low — additive only; unlocks LSTM forecast + Isolation Forest for real
5	Add quiz-answer widget in the Chat tab, wired to <code>record_answer()</code>	<code>interface/streamlit_app.py</code>	Low — additive only; unlocks DKT + DQN for real
6	Add <code>timeout=</code> to the two bare <code>requests.get</code> calls in <code>screener_adapter.py</code> ; remove the hardcoded foreign ffmpeg path in <code>stt.py</code>	Two files, two lines each	None

Phases 2–3 and 6 are pure cleanup and can happen in any order, same day. Phases 4–5 are the only ones with real design work in them, and both are additive — nothing existing has to change shape to make room for them.

25

SDE Value Assessment

Genuinely strong for an SDE interview: the module boundary discipline (every pipeline is `run(query, user_id) → dict` , swappable, testable in isolation); the fallback design in `groq_client.py` (two providers, never raises, caller-visible error state instead of a silent failure); the LP budget optimizer's infeasibility handling (`ValueError` with a human-readable reason, not a crash); the test-mocking infrastructure's correctness (patching by-value imports across nine call sites, documented why); and the routing-order bug fix documented directly in `core/orchestrator.py` 's comments — that kind of "here's the bug we hit and why the fix looks like this" comment is exactly what a strong SDE candidate's code review comments look like.

Reads as unnecessary accumulation: the five dead files in §21 — not because dead code is unforgivable, but because an interviewer who finds a broken, unreachable, un-tested file during a live walkthrough will ask why it's still there, and "leftover from an earlier version" is a weaker answer than having already removed it.

26

AI/ML Value Assessment

Genuine technical depth: six real trained models across two different families (supervised: XGBoost, LSTM×2, Isolation Forest; sequence: DKT; RL: DQN), each with its own training script, its own `is_trained()` guard, and — per your roadmap notes — real, disclosed evaluation numbers rather than cherry-picked ones (DKT's 0.74 held-out AUC below its own untested 0.80 target is documented

honestly, not hidden). The retrieval-gated tutor (confidence threshold tuned from actual score distributions, not guessed) and the strictly-grounded equity-research prompts ("use ONLY the numbers provided") are both real, specific defenses against the two most common LLM-app failure modes — hallucinated numbers and false-confidence answers.

The one real gap, stated plainly: two of the six models (DKT, DQN) and two more from the finance side (LSTM forecast, Isolation Forest) currently only get exercised at or near their cold-start/degenerate input in the live product, for the UI reasons documented in §5/§6/§20. This doesn't diminish the modeling work itself — it's a live-product integration gap, and the fix is scoped and small (§24, phases 4-5).

27

Finance/Quant Value Assessment

This is not, and doesn't present itself as, a quantitative trading system — no backtesting engine, no portfolio optimization against real market covariance, no execution logic. What it is: a personal-finance decision-support tool with real Indian tax law (sourced, dated, regime-aware), a real constrained optimizer for budget allocation, and real live market/company data grounding two of its four modules. The risk profiling is genuinely personalized (trained classifier, SHAP-explained) rather than a static questionnaire-to-label lookup table. For a BTP evaluated on "does this demonstrate real quantitative and financial reasoning, not just an LLM wrapper," the honest answer is yes on the finance and equity-research sides, and yes-but-currently-inert on the two adaptive-history models — which is a fixable, not fundamental, gap.

28

Final Product Definition

ARIA is a conversational personal finance assistant for Indian users — grounded in real deterministic financial tools (risk profiling, budget optimization, tax computation, SIP/EMI math) and a real adaptive tutor for financial literacy, narrated by an LLM that explains but never calculates, spoken and lip-synced through a free, self-hosted, browser-native 3D avatar with an optional higher-fidelity paid video mode, extended today — not someday — with real live market data and real per-company equity research, and held together by one router that sends every question to exactly the module built to answer it correctly.

29

Brutally Honest Final Verdict

You asked whether this has become over-engineered and hard to maintain. It hasn't, in the way you were worried about — there is no zoo of avatar technologies, no five-deep LLM fallback chain, no microservice sprawl, no ML bolted on for demo credibility. The actual system is smaller, cleaner, and more disciplined than the audit brief's own brainstormed component list suggested it might be. The Wav2Lip/SadTalker/MuseTalk/LivePortrait avatar-stacking scenario you asked me to check for doesn't exist in this repository, full stop.

What you actually have is a well-bounded four-module assistant where two modules (finance, tutor) each contain one real, trained, correctly-integrated ML capability that the interface never finishes connecting to — not because it's hard, but because a UI feature (transaction entry; quiz answering) was never built to close the loop. That's a smaller, more specific, and considerably more fixable problem than "this is too complex, cut things." The complexity you have is mostly earned; a handful of files are dead weight from an earlier iteration and cost you nothing to remove; and the highest-value next move isn't subtraction at all — it's two modest UI features that would make four already-real trained models visibly, demonstrably adaptive in front of an evaluator, instead of quietly frozen at their cold-start defaults.